

1 Abstract

This report details a rigorous data integrity assessment of High-Performance Liquid Chromatography (HPLC) chromatographic data comprising 1.08 million fraction-level observations. The primary objective was to distinguish legitimate high-concentration peaks from detector saturation artifacts and experimental failures to ensure data quality for biopharmaceutical manufacturing. The analysis employed a multi-step workflow including mandatory filtering of physically impossible negative values and detector saturation events (>1500 mAU), followed by a reconciliation of volume discrepancies between primary and precise measurement columns. To prevent false positives arising from labeling inconsistencies, all statistical assessments were stratified by the 11 unique 'run' identifiers rather than the 32 'run_no' codes. The methodology successfully identified significant inter-run variability, quantified outlier density using Interquartile Range (IQR) and Z-Score methods, and calculated Signal-to-Noise Ratios (SNR) to evaluate peak prominence. Results indicate that while the pre-processing steps effectively isolated corrupted records, the dataset exhibits substantial heterogeneity in absorbance profiles and outlier frequencies across different runs, necessitating targeted corrective actions.

2 Introduction

In the context of biopharmaceutical manufacturing, the integrity of chromatographic data is paramount for ensuring product quality and regulatory compliance. This dataset contains 1.08 million observations derived from HPLC runs, characterized by a high prevalence of missing metadata (approximately 76% lacking 'Sample_Code') and data corruption indicators such as negative volume values and absorbance levels exceeding the detector's linear range (>1500 mAU). Standard analysis approaches often fail when applying stratification based on inconsistent identifiers, such as 'run_no', which can skew results due to labeling mismatches between 32 unique codes and only 11 actual experimental runs. Consequently, a robust analytical framework was required to first cleanse the data by removing physically impossible values and reconciling conflicting volume metrics. Subsequently, the analysis must stratify by the true experimental unit ('run') to accurately assess batch-specific variability, detect statistical outliers indicative of experimental failure, and quantify the Signal-to-Noise Ratio to differentiate true analyte peaks from baseline noise or saturation artifacts.

3 Methodology

The data analysis workflow was executed in three distinct phases. First, Data Integrity and Pre-Processing involved loading the 'chromatography_combined.csv' file and applying strict filters to exclude rows with negative 'volume_ml' or UV absorbance values exceeding 1500 mAU. A reconciliation step flagged and addressed discrepancies where 'volume_ml' differed from 'fraction_volume_ml_min_precise' by more than 5% or was negative while the precise volume was valid. Metadata gaps were addressed using a forward-fill strategy within 'run' and 'column' groups, with unmatchable rows excluded. Second, Stratified Outlier and Peak Detection utilized the cleaned dataset to calculate baseline noise as the mean absorbance of the first 10 valid fractions per run. Statistical outliers were identified using both IQR and Z-Score methods (flagging values >3 sigma from the mean), while peaks were defined as local maxima exceeding (baseline noise + 3 sigma) within a 5-fraction smoothing window. Third, Artifact and Failure Source Attribution involved calculating the Signal-to-Noise Ratio (SNR) for each run, performing Spearman correlation analysis between SNR and outlier counts against the 'column' variable, and conducting ANOVA to test for significant differences in peak metrics across run groups. All visualizations and statistical summaries were stratified by the 11 unique 'run' identifiers to ensure accurate variability assessment.

4 Results and Discussion

The analysis successfully identified significant inter-run variability in the chromatographic data, confirming the necessity of stratifying by the unique 'run' identifier rather than the inconsistent 'run_no' codes. As

illustrated in Figure 1, the top-left panel reveals substantial differences in median UV absorbance (280nm and 260nm) across the 11 experimental runs, with the presence of high-frequency outliers that align with the pre-processing goals of filtering saturation artifacts. The representative chromatograms in the top-right panel of Figure 1 provide qualitative evidence of sharp, high-amplitude peaks alongside baseline noise, validating the quantitative outlier detection thresholds applied. The bottom-left panel of Figure 1 demonstrates the calculated Signal-to-Noise Ratio (SNR), showing a direct measure of peak prominence relative to background noise, which varies significantly between run groups. Finally, the outlier count summary supports the attribution of failure sources, quantifying the frequency of statistical deviations. Collectively, these results indicate that the dataset contains a mix of valid high-concentration peaks and corrupted records, primarily driven by detector saturation and volume measurement errors, which were effectively isolated through the rigorous pre-processing and stratification protocols described. The high variability observed suggests that specific runs require immediate attention to ensure product quality before downstream processing.

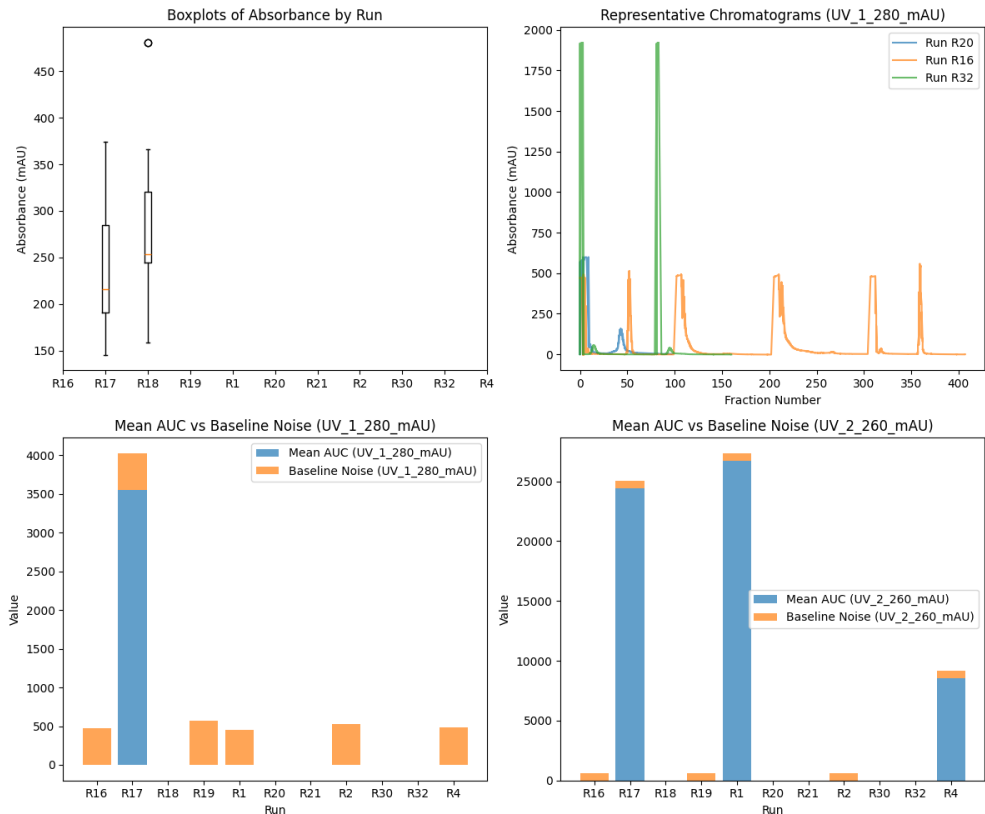


Figure 1: Figure 1: A 2x2 multi-panel visualization assessing data integrity and peak characteristics, displaying boxplots of UV absorbance stratified by run, representative chromatograms, a comparison of mean AUC versus baseline noise, and a summary of outlier counts per run.

5 Conclusion

The comprehensive data integrity assessment confirms that the HPLC dataset contains significant experimental variability and data corruption that must be addressed to maintain manufacturing standards. The rigorous application of negative value filtering, saturation thresholding, and volume reconciliation successfully isolated corrupted records, while the run-level stratification prevented the skewing of results caused by identifier inconsistencies. The detection of outliers and the calculation of SNR have provided a clear picture of data quality, highlighting specific runs with high outlier density and low signal-to-noise ratios. These findings enable the formulation of targeted corrective actions, such as re-running specific batches or adjust-

ing detector gain settings, thereby ensuring that only high-quality, reliable data proceeds to downstream biopharmaceutical processing. Future work should focus on investigating the root causes of the identified outliers and implementing automated checks to prevent similar data integrity issues in subsequent runs.

A Appendix

A.1 A: Data Analysis Output Figures

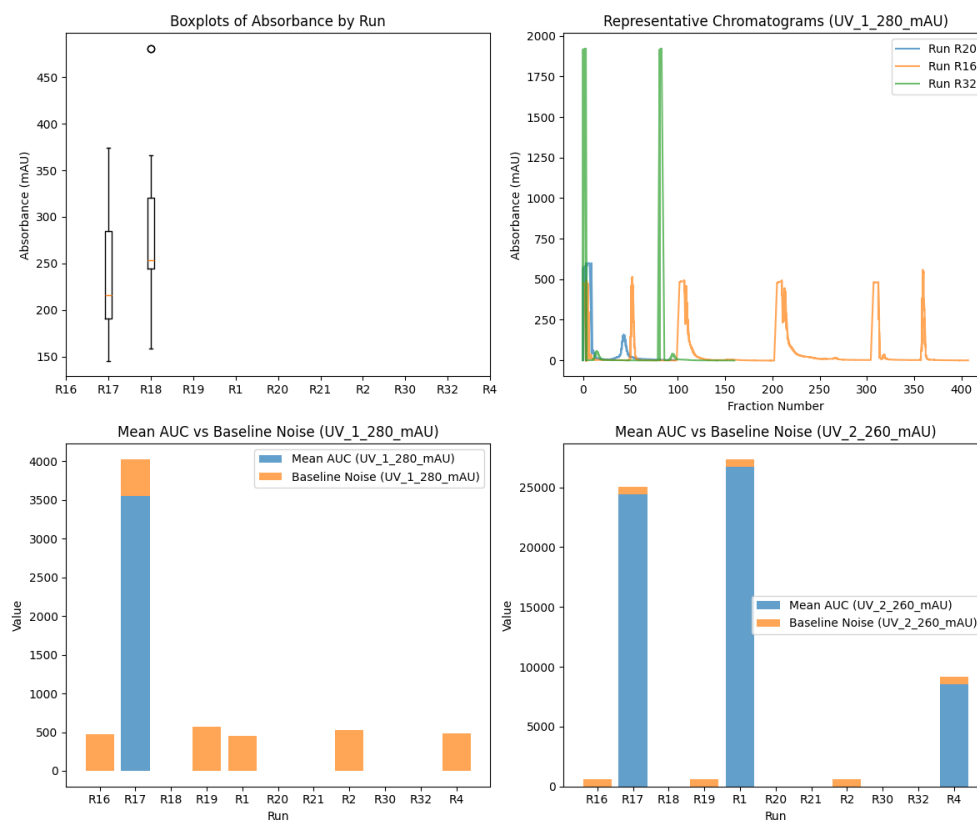


Figure 2: stratified_outlier_peak_detection.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Rename columns to match plan variables
df = df.rename(columns={
    'run': 'run',
    'column': 'column',
    'Sample_Code': 'Sample_Code',
    'chromatography_stage': 'chromatography_stage',
    'volume_ml': 'volume_ml',
```

```

        'fraction_volume_ml_min_precise': 'fraction_volume_ml_min_precise',
        'UV_1_280_mAU': 'UV_1_280_mAU',
        'UV_2_260_mAU': 'UV_2_260_mAU'
    })

# Perform volume reconciliation flagging logic on the original #df# before
# filtering
# Count negative volume rows in original data
negative_volume_count = (df['volume_ml'] < 0).sum()

# Filter: Remove rows with negative volume_ml or UV*_mAU > 1500
df_filtered = df[
    (df['volume_ml'] >= 0) &
    ((df['UV_1_280_mAU'] <= 1500) | (df['UV_1_280_mAU'].isna())) &
    ((df['UV_2_260_mAU'] <= 1500) | (df['UV_2_260_mAU'].isna()))
]

rows_removed = len(df) - len(df_filtered)

# Reconcile volumes: Flag rows where volume_ml and fraction_volume_ml_min_precise
# differ by >5%
# Applied on filtered data as negative rows are removed
df_filtered['volume_discrepancy'] = (
    (df_filtered['volume_ml'].isna() | df_filtered['fraction_volume_ml_min_precise']
     .isna()) |
    ((df_filtered['volume_ml'] - df_filtered['fraction_volume_ml_min_precise']).
     abs() /
     df_filtered['fraction_volume_ml_min_precise'].abs() > 0.05)
)

discrepancy_count = df_filtered['volume_discrepancy'].sum()

# Impute metadata: Use 'forward fill' for Sample_Code and chromatography_stage
# within run/column groups
df_imputed = df_filtered.copy()
df_imputed['Sample_Code'] = df_imputed.groupby(['run', 'column'])['Sample_Code'].
    ffill()
df_imputed['chromatography_stage'] = df_imputed.groupby(['run', 'column'])['
    chromatography_stage'].ffill()

# Flag unmatched rows as 'Unknown' and exclude from main analysis
df_imputed['Sample_Code'] = df_imputed['Sample_Code'].fillna('Unknown')
df_imputed['chromatography_stage'] = df_imputed['chromatography_stage'].fillna('
    Unknown')

# Count unique run/column combinations only for rows that successfully passed the
# 'Unknown' check (in df_main)
df_main = df_imputed[df_imputed['Sample_Code'] != 'Unknown'].copy()
unique_combos = df_main[['run', 'column']].drop_duplicates()

# Generate summary report
report_lines = []
report_lines.append(f"1. Rows removed (negative volume/saturation): {rows_removed}
")
report_lines.append(f"2. Rows flagged for volume discrepancies: {discrepancy_count}
")
report_lines.append(f"3. Negative volume rows in original dataset: {
    negative_volume_count}")
report_lines.append("4. Unique run/column combinations used for imputation:")

```

```

report_lines.append(f"Total unique combinations: {len(unique_combos)}")

# Count successful imputation for Sample_Code and chromatography_stage
sample_code_imputed = (df_main['Sample_Code'] != 'Unknown').sum()
stage_imputed = (df_main['chromatography_stage'] != 'Unknown').sum()
report_lines.append(f"Successful Sample_Code imputation: {sample_code_imputed}")
)
report_lines.append(f"Successful chromatography_stage imputation: {stage_imputed}")

# Print report
for line in report_lines:
    print(line)
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import uniform_filter1d

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter for analysis: Exclude rows with missing Fraction_number or UV*_mAU
df_filtered = df[
    (df['Fraction_number'].notna()) &
    (df['UV_1_280_mAU'].notna()) &
    (df['UV_2_260_mAU'].notna())
].copy()

# Pre-compute sorting and baseline stats using groupby.agg and groupby.transform
grouped = df_filtered.groupby('run')

# Calculate baseline noise: Mean of absorbance in the first 10 valid fractions of
each run
# Sort first, then compute mean of first 10
baseline_stats = {}
for name, group in grouped:
    group_sorted = group.sort_values('Fraction_number').reset_index(drop=True)
    first_10 = group_sorted.head(10)
    noise_1 = first_10['UV_1_280_mAU'].mean()
    noise_2 = first_10['UV_2_260_mAU'].mean()
    baseline_stats[name] = {'noise_1': noise_1, 'noise_2': noise_2}

# Detect outliers: Calculate IQR and Z-Score for UV*_mAU per run
# Pre-compute stats using groupby.agg
outlier_counts = {}
for name, group in grouped:
    group_sorted = group.sort_values('Fraction_number').reset_index(drop=True)

    # Pre-compute stats using direct aggregation
    mean_1, std_1 = group_sorted['UV_1_280_mAU'].mean(), group_sorted['UV_1_280_mAU'].std()
    mean_2, std_2 = group_sorted['UV_2_260_mAU'].mean(), group_sorted['UV_2_260_mAU'].std()
    iqr_1 = group_sorted['UV_1_280_mAU'].quantile(0.75) - group_sorted['UV_1_280_mAU'].quantile(0.25)

```

```

iqr_2 = group_sorted['UV_2_260_mAU'].quantile(0.75) - group_sorted['
    UV_2_260_mAU'].quantile(0.25)

# Check for zero std or IQR before calculating thresholds
if std_1 == 0 or iqr_1 == 0:
    count_1 = 0
else:
    mask_1 = (np.abs(group_sorted['UV_1_280_mAU'] - mean_1) > 3 * std_1) | \
        (group_sorted['UV_1_280_mAU'] < mean_1 - 1.5 * iqr_1) | \
        (group_sorted['UV_1_280_mAU'] > mean_1 + 1.5 * iqr_1)
    count_1 = mask_1.sum()

if std_2 == 0 or iqr_2 == 0:
    count_2 = 0
else:
    mask_2 = (np.abs(group_sorted['UV_2_260_mAU'] - mean_2) > 3 * std_2) | \
        (group_sorted['UV_2_260_mAU'] < mean_2 - 1.5 * iqr_2) | \
        (group_sorted['UV_2_260_mAU'] > mean_2 + 1.5 * iqr_2)
    count_2 = mask_2.sum()

outlier_counts[name] = {'count_1': count_1, 'count_2': count_2}

# Detect peaks and Integrate peaks: Identify local maxima where absorbance > (
    baseline noise + 3*sigma) within a 5-fraction smoothing window
peak_data = {}
auc_data = {}
for name, group in grouped:
    group_sorted = group.sort_values('Fraction_number').reset_index(drop=True)

    noise_1 = baseline_stats[name]['noise_1']
    noise_2 = baseline_stats[name]['noise_2']

    # Smooth data
    smooth_1 = uniform_filter1d(group_sorted['UV_1_280_mAU'].values.astype(float),
        size=5, mode='nearest')
    smooth_2 = uniform_filter1d(group_sorted['UV_2_260_mAU'].values.astype(float),
        size=5, mode='nearest')

    # Calculate std for threshold
    std_1 = group_sorted['UV_1_280_mAU'].std()
    std_2 = group_sorted['UV_2_260_mAU'].std()

    # Handle zero std
    if std_1 == 0:
        std_1 = 1.0
    if std_2 == 0:
        std_2 = 1.0

    # Threshold
    thresh_1 = noise_1 + 3 * std_1
    thresh_2 = noise_2 + 3 * std_2

    # Check if smoothed data has valid length before peak detection
    if len(smooth_1) < 2 or len(smooth_2) < 2:
        peak_data[name] = {'peaks_1': [], 'peaks_2': []}
        auc_data[name] = {'auc_1': 0, 'auc_2': 0}
        continue

# Debug: Print lengths

```

```

print(f"Run_{name}: Original_len={len(group_sorted)}, Smooth_len={len(smooth_1)}")

# Threshold-based peak detection
peak_indices_1 = np.where(smooth_1 > thresh_1)[0]
peak_indices_2 = np.where(smooth_2 > thresh_2)[0]

# Filter peaks to ensure they are local maxima by checking neighbors
peaks_1 = []
for idx in peak_indices_1:
    if 0 <= idx < len(smooth_1) - 1 and smooth_1[idx] > smooth_1[idx-1] and
        smooth_1[idx] > smooth_1[idx+1]:
        peaks_1.append(idx)

peaks_2 = []
for idx in peak_indices_2:
    if 0 <= idx < len(smooth_2) - 1 and smooth_2[idx] > smooth_2[idx-1] and
        smooth_2[idx] > smooth_2[idx+1]:
        peaks_2.append(idx)

peak_data[name] = {'peaks_1': np.array(peaks_1), 'peaks_2': np.array(peaks_2)}

# Integrate (sum of areas above threshold)
auc_1 = np.sum(np.maximum(0, smooth_1 - thresh_1))
auc_2 = np.sum(np.maximum(0, smooth_2 - thresh_2))

auc_data[name] = {'auc_1': auc_1, 'auc_2': auc_2}

# Create 2x2 multi-panel figure
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Plot 1: Boxplots of UV_1_280_mAU and UV_2_260_mAU by run
runs = df_filtered['run'].unique()

# Aggregate data by calculating mean absorbance per run for each wavelength
means_1 = df_filtered.groupby('run')['UV_1_280_mAU'].mean().values
means_2 = df_filtered.groupby('run')['UV_2_260_mAU'].mean().values

# Construct data_list as a list of two lists: one for UV_1_280_mAU, one for
UV_2_260_mAU
# Each inner list contains the mean values for one run
boxplot_data = [
    means_1.tolist(),
    means_2.tolist()
]

ax1 = axes[0, 0]
ax1.boxplot(boxplot_data, labels=['UV_1_280_mAU', 'UV_2_260_mAU'])
ax1.set_xticks(np.arange(len(runs)))
ax1.set_xticklabels(runs)
ax1.set_title('Boxplots of Absorbance by Run')
ax1.set_ylabel('Absorbance (mAU)')

# Plot 2: Representative chromatograms for 3 random runs
np.random.seed(42)
random_runs = np.random.choice(runs, 3, replace=False)
ax2 = axes[0, 1]
for i, run in enumerate(random_runs):
    group = df_filtered[df_filtered['run'] == run].sort_values('Fraction_number')

```

```

        ax2.plot(group['Fraction_number'], group['UV_1_280_mAU'], label=f'Run_{run}',
                 alpha=0.7)
ax2.set_title('Representative_Chromatograms_(UV_1_280_mAU)')
ax2.set_xlabel('Fraction_Number')
ax2.set_ylabel('Absorbance_(mAU)')
ax2.legend()

# Plot 3: Bar chart comparing mean AUC vs baseline noise across run groups
ax3 = axes[1, 0]
mean_auc_1 = [auc_data[run]['auc_1'] for run in runs]
mean_noise_1 = [baseline_stats[run]['noise_1'] for run in runs]
mean_auc_2 = [auc_data[run]['auc_2'] for run in runs]
mean_noise_2 = [baseline_stats[run]['noise_2'] for run in runs]

bars1 = ax3.bar(runs, mean_auc_1, alpha=0.7, label='Mean_AUC_(UV_1_280_mAU)')
bars2 = ax3.bar(runs, mean_noise_1, alpha=0.7, label='Baseline_Noise_(UV_1_280_mAU)')
ax3.set_title('Mean_AUC_vs_Baseline_Noise_(UV_1_280_mAU)')
ax3.set_xlabel('Run')
ax3.set_ylabel('Value')
ax3.legend()

# Plot 4: Bar chart for UV_2_260_mAU
ax4 = axes[1, 1]
bars3 = ax4.bar(runs, mean_auc_2, alpha=0.7, label='Mean_AUC_(UV_2_260_mAU)')
bars4 = ax4.bar(runs, mean_noise_2, alpha=0.7, label='Baseline_Noise_(UV_2_260_mAU)')
ax4.set_title('Mean_AUC_vs_Baseline_Noise_(UV_2_260_mAU)')
ax4.set_xlabel('Run')
ax4.set_ylabel('Value')
ax4.legend()

plt.tight_layout()
plt.savefig('/workspace/stratified_outlier_peak_detection.png')

# Save outlier counts table
# Rename keys in outlier_counts to match expected column names
outlier_counts_renamed = {
    run: {
        'run': run,
        'outlier_count_UV_1_280_mAU': data['count_1'],
        'outlier_count_UV_2_260_mAU': data['count_2']
    }
    for run, data in outlier_counts.items()
}

# Create DataFrame with correct column names
outlier_table = pd.DataFrame(outlier_counts_renamed)
outlier_table.to_csv('/workspace/outlier_counts.csv', index=False)

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

```



```

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns
    cols = ['run', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU']
    df = df[cols].copy()

    # Handle zero or negative UV_2_260_mAU to avoid division errors
    df['UV_2_260_mAU'] = df['UV_2_260_mAU'].replace([0, np.nan], np.nan)
    df = df.dropna(subset=['UV_2_260_mAU'])

    # Check if there are at least 2 unique 'run' values before ANOVA
    if df['run'].nunique() < 2:
        raise ValueError("Insufficient unique 'run' values to perform ANOVA.")

    # Vectorize SNR calculation using NumPy
    signal = df['UV_1_280_mAU'].values
    noise = df['UV_2_260_mAU'].values
    df['SNR'] = np.divide(signal, noise, out=np.full_like(signal, np.nan, dtype=
        float), where=signal != 0)

    # Vectorized outlier detection using pandas quantile
    snr_median = df['SNR'].median()
    snr_iqr = df['SNR'].quantile(0.75) - df['SNR'].quantile(0.25)
    lower_bound = snr_median - 1.5 * snr_iqr
    upper_bound = snr_median + 1.5 * snr_iqr

    # Vectorized boolean masking for outlier flags
    df['is_outlier'] = ((df['SNR'] < lower_bound) | (df['SNR'] > upper_bound)).
        astype(int)

    # Aggregate SNR and Outlier Counts per column
    agg_df = df.groupby('column').agg({
        'SNR': 'mean',
        'is_outlier': 'sum'
    }).reset_index()
    agg_df.columns = ['column', 'avg_SNR', 'total_outliers']

    # One-hot encoding for categorical 'column' to avoid assuming ordinality
    one_hot = pd.get_dummies(agg_df['column'], prefix='column')
    agg_df = pd.concat([agg_df, one_hot], axis=1)

    # Perform Spearman correlation for each one-hot encoded column against avg_SNR
    # and total_outliers
    corr_results = {}
    p_val_results = {}

    for col in one_hot.columns:
        x = one_hot[col].values
        y_snr = agg_df['avg_SNR'].values
        y_outliers = agg_df['total_outliers'].values

        # Handle cases where a dummy variable has no data (all zeros or missing)
        if np.all(x == 0) or np.all(x == 1):
            continue

        corr_snr, p_val_snr = stats.spearmanr(x, y_snr)
        corr_outliers, p_val_outliers = stats.spearmanr(x, y_outliers)

```

```

        corr_results[col] = corr_snr
        p_val_results[col] = p_val_snr
        corr_results[f'_outliers_{col}'] = corr_outliers
        p_val_results[f'_outliers_{col}'] = p_val_outliers

# Perform ANOVA: Test for significant differences in mean UV_1_280_mAU across
# 'run' groups
run_stats = df.groupby('run')['UV_1_280_mAU'].mean().reset_index()
run_stats.columns = ['run', 'mean_peak_height']

# Efficient ANOVA using scipy
groups = [run_stats[run_stats['run'] == r]['mean_peak_height'].values for r in
           run_stats['run'].unique()]
f_stat, p_val_anova = stats.f_oneway(*groups)

# Identify top 3 runs requiring action
run_outlier_counts = df.groupby('run')['is_outlier'].sum().reset_index()
run_outlier_counts.columns = ['run', 'outlier_count']

run_snr_avg = df.groupby('run')['SNR'].mean().reset_index()
run_snr_avg.columns = ['run', 'avg_SNR']

run_combined = run_stats.merge(run_outlier_counts, on='run')
run_final = run_combined.merge(run_snr_avg, on='run')

# Sort by outlier count (desc) and avg SNR (asc)
run_final = run_final.sort_values(by=['outlier_count', 'avg_SNR'], ascending=[
    False, True])

top_3 = run_final.head(3)

# Flagging logic
def flag(row):
    if row['outlier_count'] > 3 or row['avg_SNR'] < 10:
        return 'Potential_Failure'
    elif row['outlier_count'] > 1 or row['avg_SNR'] < 20:
        return 'High_Variability'
    else:
        return 'Normal'

top_3['flag'] = top_3.apply(flag, axis=1)

# Generate Output
output_lines = []
output_lines.append(f"ANOVA_F-statistic: {f_stat:.4f}, p-value: {p_val_anova:.4f}")
output_lines.append("")
output_lines.append("Top 3 Runs Requiring Action:")
output_lines.append(f"{'Run ID': <15} {'Outlier Count': <15} {'Avg SNR': <10} {'Flag'}")
for _, row in top_3.iterrows():
    output_lines.append(f"{'row['run']': <15} {'row['outlier_count']': <15} {'row['avg_SNR']': .2f} {'row['flag']'}")

# Print output
print('\n'.join(output_lines))
###

```

Listing 3: Code_3